

QUERY 6 INDEX ANALYSIS

OBJECTIVE :

Analyze TPC-H Query 6 to identify opportunities for index selection. Creating appropriate indexes and demonstrating performance improvements through comparative analysis.

TPC-H Query 6 :

The query lists the amount by which the total revenue would have increased if these discounts had been eliminated for `lineitems` with `l_quantity` less than quantity. Note that the potential revenue increase is equal to the sum of `[l_extendedprice * l_discount]` for all `lineitems` with discounts and quantities in the qualifying range.

Query Specification :

```
EXPLAIN (ANALYZE, FORMAT JSON)
SELECT SUM(l_extendedprice * l_discount) AS revenue
FROM lineitem
WHERE l_shipdate >= DATE '1994-01-01'
      AND l_shipdate < DATE '1995-01-01'
      AND l_discount BETWEEN 0.05 AND 0.07
      AND l_quantity < 24;
```

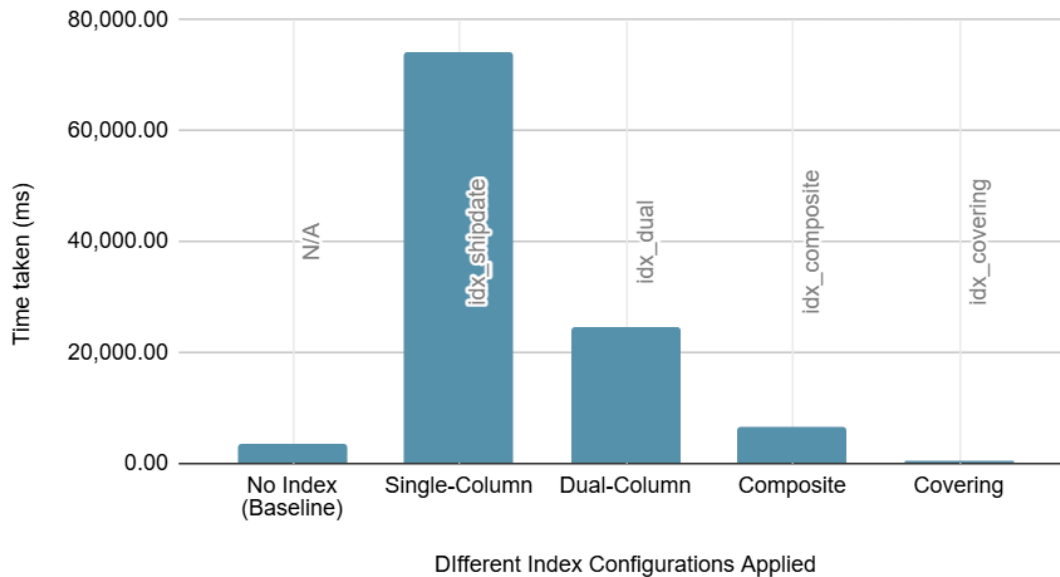
Filter	Selectivity	Cardinality
<code>l_shipdate >= '1994-01-01' AND l_shipdate < '1995-01-01'</code>	14%	856332
<code>l_discount BETWEEN 0.03 AND 0.05</code>	30%	1799915
<code>l_quantity < 24</code>	46%	2817735

Now we will try the following configurations and perform comparative analysis :

1. Without Indexes
2. Creating index on (`l_shipdate`)
3. Creating composite index on (`l_shipdate`, `l_discount`)
4. Creating composite index on (`l_shipdate`, `l_discount`, `l_quantity`)
5. Creating a Covering Index (`l_shipdate`, `l_discount`, `l_quantity`) **INCLUDE** (`l_extendedprice`)

Case	Strategy	Index Name	Plan Type	Execution Time (ms)	Speedup vs. Seq Scan
0	No Index (Baseline)	N/A	Seq Scan	3,475.81	1.00x
1	Single-Column	idx_shipdate	Index Scan	74,128.88	0.05x Slower
3	Dual-Column	idx_dual	Index Scan	24,529.08	0.14x Slower
4	Composite	idx_composite	Index Scan	6,473.99	0.54x Slower
5	Covering	idx_covering	Index Only Scan	582.67	5.96x faster

Query Execution Performance



1. Without Indexing (Sequential Processing)

- Ran the query with no indexes present.
- The database engine performs a **Seq Scan** (Sequential Scan), reading every page of the **lineitem** table from start to finish reading the table linearly is relatively efficient because it utilizes "Sequential I/O," which the hardware handles well. However, it still must filter and discard out nearly 11.7 million rows manually to find the target data.
- **Time Taken: 3.47 seconds.**

Candidate Search Keys :

Based on the where clause in the SQL query we can identify the following potential candidate keys

1. **l_shipdate** :

Was considered a good starting point as it is primary condition in the WHERE clause

- Used in a range predicate
- Highly selective (1 year out of many)
- Very common filter in TPC-H

2. **l_discount**

- Narrow numeric range
- Medium selectivity

3. **l_quantity**

- Upper-bound predicate
- Medium selectivity

4. **Combinations of the above**

- (**l_shipdate**, **l_discount**)
- (**l_shipdate**, **l_discount**, **l_quantity**)

2. Single - Column Indexing :

A single-column index on **l_shipdate** was evaluated as a baseline candidate since it is the most selective predicate and appears in all instances of the query.

- The engine uses the index to find rows within the 1994 date range.
- **Time Taken : 74.12 seconds.**
- While the index quickly finds the dates, it doesn't contain the **l_extendedprice**. The database must "jump" to the actual table (the Heap) for every single match. These millions of "jumps" create **Random I/O**, causing it to be much slower than running without indexes.

3. Dual - Column Composite Index :

Reasoned that by narrowing search criteria and conditions in the index, we would be able to reduce the number of rows database fetches from Heap. Created **idx_dual** on (**l_shipdate**, **l_discount**).

- **Logic:** The index now filters by both date and discount range before looking at the table.
- **Time Taken : 24.52 seconds.**
- Despite initial reasoning that a narrower index will lead to lesser Random I/O accesses, ever, it still needs to visit the Heap for the **l_extendedprice**, so Random I/O remains a bottleneck, though less severe than in Case 2.

4. A Composite Index on (l_shipdate**, **l_discount**, **l_quantity**)**

Reasoned that would improve performance evaluated since all three columns appear as range predicates in the WHERE clause and are always used together.

- **Time Taken: 6.47 seconds.**
- The engine identifies the exact rows needed entirely within the index. It only visits the table to retrieve the final value (**l_extendedprice**). However, it is still slower than running without any indexes as overhead of jumping to the table is still higher than a simple linear scan.

5. Covering Index :

Reasoned that the created `idx_covering` using `INCLUDE (l_extendedprice)` would allow the databases to find all parameters to execute query (the filters and the value to sum) in one place.

- This creates an **Index Only Scan**.
- **Time Taken : 0.58 seconds (582 ms)**.
- It performs zero Heap Fetches, eliminating Random I/O entirely. This results in a 127x improvement over the single-column index and a 6x improvement over the baseline.

CONCLUSION :

The progression of indexes experimented with shows that for high-volume TPC-H queries, standard indexes may hurt performance due to the cost of table lookups (Random I/O). Bypassing the table entirely using a **Covering Index** was proven to be the best method to reduce latency of running the query.

Experimental Setup:

Dataset used: TPC-H Database (10GB)

System Configuration:

- Operating System: Windows 64-bit (x64-based processor)
- Processor (CPU) : Intel Core i7-10510U @ 1.80 GHz
- Number of Cores: 4
- RAM: 16 GB (15.7 GB usable)
- Storage: Solid State Drive (SSD)
- Database Management System: PostgreSQL 18
- Workload Type (pgTune profile): Data Warehouse

pgTune Parameters:

name	setting	unit	source
checkpoint_completion_target	0.9		configuration file
default_statistics_target	500		configuration file
effective_cache_size	1572864	8kB	configuration file
huge_pages	off		configuration file
maintenance_work_mem	2096128	kB	configuration file
max_connections	40		configuration file
max_parallel_maintenance_workers	2		default
max_parallel_workers	8		default
max_parallel_workers_per_gather	2		default
max_wal_size	16384	MB	configuration file
max_worker_processes	8		default
min_wal_size	4096	MB	configuration file
random_page_cost	1.1		configuration file
shared_buffers	524288	8kB	configuration file
wal_buffers	2048	8kB	configuration file
work_mem	43690	kB	configuration file
(16 rows)			